Succession is not a closed loop

A comparison of Turing's machines, Bennett's logical reversibility, and a later composition rule

Abstract

Turing's 1936 account of computation is a theory of successive complete configurations of a machine with finitely many states and one scanned square. A number can be defined and fail to be computable, and a machine is circular, in his sense, when it prints only finitely many figures, not when its causal graph contains a cycle. Bennett's 1973 construction shows that an ordinary Turing machine can be emulated by a machine whose transition function has a single-valued inverse at every step, by saving a history, copying the output, and then uncomputing the history. That reversibility is still reversibility of a succession. A process function, in a literature that begins decades later, is a different object: a wiring in which every replacement of the local maps has exactly one global history. Its canonical closed-loop operator is unitary for every local unitary and every matched ancilla if and only if that condition holds, while a bare permutation of a source-sink space exists for wirings that are not process functions. No influence from Turing or Bennett to that later formalism is claimed or documented here. The comparison is a rational reconstruction of three jobs. It is not a story in which each result completes the previous one.

Keywords: Turing machines; logical reversibility; process functions; rational reconstruction

1 What this comparison is

The finite results used below are already public. They are not archival findings, and they are not offered as new theorems. The historical claim is narrower than a history of computation. It is an argument that three reversibility-like ideas, often placed in one sentence, do different work, and that the differences can be read off the texts without inventing a line of descent.

The first text is Turing's "On Computable Numbers, with an Application to the Entscheidungsproblem," read from the London Mathematical Society printing (Turing 1936). The second is Bennett's "Logical Reversibility of Computation," read from the IBM Journal printing (Bennett 1973). The third is the definition of a finite process function and the canonical unitary lift, taken from the published mathematical literature that states them (Baumeler and Wolf 2016; Oreshkov, Costa, and Brukner 2012; Stinespring 1955), not from an unpublished notebook. Landauer is present only as Bennett cites him. Landauer's own paper was not re-read for this draft, and nothing is quoted from it.

A rational reconstruction, as the phrase is used here, is an argument that sets out what each text is doing and what follows if one asks a later question of an earlier text. It is not evidence that the later author had the earlier text in mind. An influence claim would need a citation, a letter, a shared technical vocabulary used as such, or some other trace. None is produced in what follows. The absence of a trace in this reading is not a discovery that no trace exists.

There is also no progress narrative. Bennett does not repair a defect in Turing that Turing had posed as the problem of closed causal loops. Process functions do not complete Bennett's thermodynamic program. Each text answers a question its author states. The questions are not the same question.

2 Turing: succession under a finite memory

Turing's opening description is brief and operational. The computable numbers "may be described briefly as the real numbers whose expressions as a decimal are calculable by finite means." A number is computable, on the definition he then adopts, if its decimal can be written down by a machine. He says at once that the computable numbers "do not, however, include all definable numbers," and that an example of a definable number that is not computable will be given. Definability and computability are separated in the first pages.

A later composition rule that happens to be finitely checkable does not inherit the prestige of that separation, and a rule that fails a consistency test is not thereby one of Turing's non-computable numbers.

The justification he offers for the machine, before the formal development, is that "the human memory is necessarily limited." He compares a person computing a real number to a machine capable of finitely many conditions, which he calls m-configurations. The machine has a tape divided into squares. At any moment one square is scanned. "The scanned symbol is the only one of which the machine is, so to speak, 'directly aware'." By changing its m-configuration the machine can remember some of the symbols it has scanned. The possible behaviour at a moment is fixed by the pair of m-configuration and scanned symbol, and that pair is the configuration. The machine may write, erase, and shift the scanned square by one place. Symbols that are not part of the decimal are "rough notes to 'assist the memory'," and only those notes are liable to erasure.

Two definitions in the second section matter for the comparison. An automatic machine is one whose motion at each stage is completely determined by the configuration. A computing machine prints figures, which are 0 and 1, and other symbols. Started from the correct initial m-configuration on a blank tape, the figures it prints are the sequence computed by the machine. The complete configuration at a stage is the number of the scanned square, the whole sequence of symbols on the tape, and the m-configuration. "The changes of the machine and tape between successive complete configurations will be called the moves of the machine." Computation, in this text, is a sequence of moves. There is a before and an after at every step. Nothing in the definition of a move is a constraint on a system of equations that are all imposed at once.

The circular machines are the point at which a later reader is most tempted to import a different idea. Turing's definition is about printing. If a computing machine "never writes down more than a finite number of symbols of the first kind, it will be called circular. Otherwise it is said to be circle-free." He adds that a machine is circular if it reaches a

configuration from which there is no possible move, or if it continues and perhaps prints symbols of the second kind but can print no further figures. He postpones the significance of the word to §8. In §8 the diagonal argument is used to show that there is no general process for telling whether a description number is the description number of a circle-free machine. A number that describes a circle-free machine is a satisfactory number, and "there can be no general process for determining whether a given number is satisfactory." The word "circular" is tied, in the text, to the failure of an infinite printout and to the diagonal construction, not to a cycle in a causal graph. A wiring in which three bits depend on one another is not a circular machine in Turing's sense. A machine that prints 010101 . . ., his first example, is not a causal loop because the digits alternate.

The universal machine is also a machine of succession. "It is possible to invent a single machine which can be used to compute any computable sequence." If that machine is supplied with a tape on which the standard description of another computing machine has been written, it computes the same sequence. The description sits on the tape. The universal machine consults it while producing successive complete configurations. Universality here is the existence of one table that can follow any other table presented as symbols. It is not the claim that every consistent system of local maps has a unitary dilation, and it is not the claim that an inconsistent system is uncomputable. An inconsistent finite wiring can still be handed to a Turing machine as a finite table. The machine can search the table. What it cannot do, if the wiring has no history, is return a history. That failure is a failure of the search's specification, not a specimen of Turing's non-computable numbers. The search halts and reports that the finite set is empty.

Turing's §9 and §11 are not re-argued here. The paper's application to the Entscheidungsproblem is part of what the 1936 text is for. Using the text as a source about closed causal composition, while leaving that application aside, is legitimate only if the use is labeled as a later question. The later question is not one Turing poses. He is not, in the pages read, classifying physical experiments by whether their local interventions have unique joint

outcomes.

3 What §9 adds, and what it still does not say

Section 9 of the 1936 paper is Turing's argument that the machines he defined are broad enough. It is the place a later reader might look for a thesis about physical realization. What it actually does is constrain the computer's steps.

He supposes that the number of symbols and the number of states of mind that need be taken into account are finite. An infinity of states of mind, he says, would include states "arbitrarily close" that would be confused. More complicated states of mind can be avoided by writing more symbols on the tape. He then splits the computer's work into simple operations, each a change of the system consisting of the computer and his tape. The state of that system is fixed by the symbols on the tape, which of them are observed, and the state of mind. In a simple operation not more than one symbol is altered. A change of the observed squares may move only to squares within a fixed distance L of a previously observed square. He illustrates the point about immediate recognition with a numbered theorem whose number is short enough to take in at a glance, and a number long enough that the computer must compare it figure by figure. Recognition of a long number is not a simple operation.

The simple operations therefore include a change of one observed symbol, possibly with a change of state of mind, and a change of an observed square to another within L squares, possibly with a change of state of mind. The operation performed is determined by the state of mind and the observed symbols. To each state of mind there corresponds an m-configuration. Turing's claim is that a computing machine in the sense of §2 can be constructed to compute the same sequence. The argument enlarges the informal picture of a human computer and then reduces it to the earlier machine. It does not replace succession by a simultaneous constraint. Each simple operation is still one step, and the next state of mind is determined by the present one together with the observed symbols.

The third part of §9 makes the same structure without calling the note a state of mind. The computer may break off, leave, and return, provided he leaves "a note of instructions (written in some standard form) explaining how the work is to be continued." Turing supposes the computer never does more than one step at a sitting. The note must enable him to carry out one step and write the next note. "The state of progress of the computation at any stage is completely determined by the note of instructions and the symbols on the tape." He calls the resulting expression a state formula. The state formula at a stage is determined by the state formula before the last step. He assumes that this relation can be expressed by an axiom in the functional calculus, and then a machine can write down the successive state formulae.

That is as close as the paper comes to a slogan about implementation, and the slogan is still about a function from one complete state to the next. A process function has no state formula and no next step inside its definition. The fixed-point condition says which output tuples agree with the local maps. One can write the wiring down as a finite table and ask a Turing machine to search it. The search has state formulae. The wiring does not acquire them. Section 9 therefore supports the comparison in one direction only. It shows why Turing's notion of computation is a notion of successive, locally determined changes. It does not show that every mathematically well-defined relation among variables is a computation in his sense, and it does not show that a relation without a unique solution is one of his non-computable sequences. The sequence δ he defines in §9, whose nth figure records whether n is satisfactory, is not computable because there is no uniform process for the satisfactory numbers. A finite wiring with no history is a finite set that a uniform search can certify as empty.

The second part of §9, on definability in the functional calculus, is the other half of the opening distinction. A formula $\mathfrak{A}$ "defines" a sequence when $\neg\mathfrak{A}$ is not provable and, for each n, one of two formulae giving the nth figure is provable. Turing then says that the computable numbers do not include all numbers definable "in the ordinary sense," and he

points to δ. Ordinary definability outruns the machines. The later composition rule is not an entry in that gap. It is a finite condition, decidable by search, about maps on finite alphabets. Placing it among Turing's definable-but-not-computable numbers would be a category error, not a historical continuation.

4 Bennett: a single-valued inverse at every step

Bennett's abstract states the thesis in one sentence. "The usual general-purpose computing automaton (e.g., a Turing machine) is logically irreversible—its transition function lacks a single-valued inverse." He then says such machines may be made logically reversible at every step while keeping their ability to do general computations. The physical interest he claims is the plausibility of thermodynamically reversible computers that dissipate considerably less than kT of energy per logical step. This draft does not assess that thermodynamic claim. It uses the logical construction, which is fully stated in the article, and it treats the thermodynamic moral as Bennett's, not as a lemma about process functions.

The introduction distinguishes the operations that throw away information about a computer's history: erasure, overwriting, and entry into a part of a program addressed by several transfers. The typical computer's transition function, "the partial function that maps each whole-machine state onto its successor, if the state has a successor," lacks a single-valued inverse. Bennett reports Landauer as having asked whether this is unavoidable in useful computers, and as having argued that a physical computer that throws away information about its previous state must generate entropy, at least $kT \ln 2$ per erased bit. Those sentences are Bennett's report. They are not an independent reading of Landauer, and they are not transferred here into a claim about the entropy of a process matrix.

Bennett's requirement on a useful reversible computer is easy to miss if one stops at "save the history." An irreversible computer can be made reversible by recording everything it would have thrown away. He says it is reasonable to demand, further, that if the machine

halts it should have erased its intermediate results, leaving the desired output and the original input. The input must be kept, "otherwise it could not be reversible and still carry out computations in which the input was not uniquely determined by the output." Reversibility of the step and uniqueness of a function from output back to input are not the same demand. He allows the machine to remember the input precisely because many computations are not injective.

The construction has three stages, described in the abstract and again in the introduction. In the first, the reversible automaton parallels the irreversible one but saves intermediate results, avoiding erasure. In the second, it prints the desired output. In the third, it disposes of the intermediate results by retracing the first stage backward, which is possible because the first stage was reversible, and restores the machine, apart from the written output, to its original condition. The final configuration contains the output and a reconstructed copy of the input. A history tape full of arbitrary data could not be erased reversibly. The history tape produced by the computation is not arbitrary: it is redundant with the machine that wrote it, so the inverse transitions can blank it. Computations on the reversible computer "take about twice as many steps" as on an ordinary one and may need a large amount of temporary storage.

The formal criterion is stated for quintuples and then for his quadruples. Each quintuple of the ordinary read-write-shift kind defines a partial one-to-one map of the whole-machine state onto its successor, and is itself deterministic and reversible. A Turing machine is deterministic if and only if its quintuples have non-overlapping domains, and reversible if and only if they have non-overlapping ranges. The usual machine fails the second test. Bennett's quadruples are arranged so that a transition and its inverse have the same form: on each tape the machine either reads and writes or shifts, not both. The three-tape machine he builds from an ordinary machine S emulates S and, at the end, leaves only the input and the output. The emulation is step-by-step. The reversible machine still has successive states.

Two limits of this reading should be marked. First, Bennett discusses messenger RNA

as a physical example of reversible computation. That discussion is not used below. It is not evidence about process functions, and this paper does not reinterpret molecular biology. Second, the phrase "logical reversibility" in Bennett's title means the existence of a single-valued inverse of the transition function, together with the tidiness condition on the halted tape. It does not mean that a closed loop of local unitaries is unitary, and it does not mean that every substitution of local functions has one fixed point.

5 A later composition rule

The process-function literature is later by decades. Nothing in the 1936 or 1973 texts uses its vocabulary, and this section does not pretend that the vocabulary was latent in them. The objects are stated so that the comparison has a definite target.

A finite wiring assigns to each party k an input alphabet I_k and an output alphabet O_k, and a map w_k from the tuple of all outputs to I_k. A local function is a map $f_k \colon I_k \to O_k$. The wiring is a process function when, for every tuple (f_k), the map sending an output tuple o to $(f_k(w_k(o)))$ has exactly one fixed point. The fixed point is the history. The quantifier is universal over local functions. One history for one choice of local functions is not enough.

Three small wirings show that existence, uniqueness, and order come apart. They are calculations, not historical episodes. In the first, two parties share a bit, one applies the identity channel, and the other applies a bit flip. No pair of outputs satisfies both local maps. Existence fails. In the second, the second party copies the bit instead of flipping it. Both bit values are histories. Uniqueness fails. In the third, the three-bit rule $w(a, b, c) = (\neg b \wedge c,\ \neg c \wedge a,\ \neg a \wedge b)$ has, for the three copying functions, only the history $(0, 0, 0)$, and the same uniqueness holds for every other substitution of the four binary functions. The third wiring is cyclic in the ordinary graph sense: each bit depends on the others. It is a process function. Turing's word "circular" does not apply to it, because nothing in the example is a machine printing figures.

The canonical lift is a unitary operator built from the wiring, acting on a space that

records outputs and a matched ancilla. It is unitary for every local unitary and every finite ancilla whose dimensions match the input and output alphabets if and only if the wiring is a process function. The proof is a counting argument: when the local maps are unitary dilations of functions, a column of the closed loop has squared norm equal to the number of fixed points, so the loop is unitary for all such choices exactly when that number is always one. The details are in the public mathematical drafts. They are not re-proved here, and no new case is claimed.

A different operator is sometimes confused with this lift. For an arbitrary function w from outputs to inputs one can define a permutation of a larger space by $U|o, e\rangle = |w(o) + e, o\rangle$, with addition in the ancilla alphabet. That permutation exists whether or not any fixed point exists. It is a basis permutation for any w. It does not say that the wiring has a history, and it is not an implementation of the wiring as a machine that runs. Bennett's reversible machine is an implementation in a precise sense: it emulates the irreversible machine step by step and halts with input and output. The basis permutation does not emulate a Turing machine, reversible or otherwise. It relabels a basis.

The known numerical witness that some process functions produce a negative value of a particular linear functional, in one computational basis, on a known board, is likewise not a historical object. It is a finite table. The table can be searched. The search is a Turing-machine task of the most ordinary kind. Its existence shows that the witness is not a non-computable number. It does not show that Turing anticipated the table, and it does not show that the search is the physical layout of the board.

6 Where the three jobs refuse to line up

The first refusal is about time order. Turing's move takes one complete configuration to the next. Bennett's quadruple does the same, with the added demand that the map on whole-machine states be invertible and that the halted tape be tidy. A process function is

a condition on a system of maps that are applied together. The history is a fixed point of the composition of the wiring with the local functions. There is no scanned square and no next m-configuration inside that definition. One can, if one wants, write a Turing machine that searches for the fixed point. That machine has moves. The wiring's definition does not become a sequence of moves because a search for its histories has moves. The search is a further model.

The second refusal is about the word "circular." Importing Turing's term into a cyclic wiring is a verbal accident. His circular machine fails to print infinitely many figures, or gets stuck, or is the diagonal object in §8. A cyclic dependence among three bits can be a process function, as the third wiring is, and a machine with an entirely linear tape can be circular in his sense by printing only finitely many figures. The two uses of the word do not mark one phenomenon.

The third refusal is about reversibility. Bennett's reversible machine has non-overlapping ranges, so each whole-machine state has at most one predecessor. The canonical lift is unitary on a complex space of outputs and ancillas, for every local unitary, precisely when every substitution has one history. These are not two descriptions of one theorem. A wiring can fail to be a process function and still define Bennett's bare permutation of the source-sink space. The identity-and-copy wiring is the example: two histories, so not a process function, while the basis permutation of its w still exists. Conversely, the three-bit process function has a unitary canonical lift, but that fact does not mean that a Turing machine computing something else has been replaced by a reversible Turing machine in Bennett's construction. No history tape is written, and no third stage uncomputes it.

The fourth refusal is about universality. Turing's universal machine follows a standard description written on its tape and produces the sequence the described machine would produce. Universality is emulation of any circle-free computing machine. The canonical lift is not universal in that sense. It is defined for one wiring at a time, and the theorem says for which wirings the lift is unitary under all local unitaries. A process function on three

bits does not compute arbitrary computable sequences. Saying that the lift is a universal computer because the word "universal" appears in Turing, or because the consistency condition quantifies over all local maps, confuses a quantifier with a universal machine.

The fifth refusal is about thermodynamic moral. Bennett's interest in dissipation is explicit, and it rests on Landauer's argument as Bennett gives it. The process-function theorem does not mention entropy, temperature, or erasure of a bit. A unitary operator does not, by itself, tell one what heat a laboratory device would produce. To move from the theorem to a dissipation claim one would need a physical model of the device and an argument of Landauer's kind. Neither is supplied by the existence of the lift. This paper does not supply them either.

The sixth refusal is about decidability. Turing shows there is no general process for the satisfactory numbers. Finite process functions on explicit tables are outside that negative result. The set of substitutions is finite, the set of candidate histories is finite, and a search decides the condition. The decision procedure is not a deep consequence of Turing's paper. It is the triviality Turing's framework assigns to finite tables. Treating the search as a new complexity theorem, or as a hypercomputation, would invert the point. The search also does not implement the wiring. It implements the search.

7 Influence would be a different paper

It is possible to write a history in which Bennett is a reader of Turing and the process-function authors are readers of Bennett. Parts of that history are obviously true in a thin sense: Bennett cites a Turing machine as his example of a computing automaton, and his reference list points at Turing's paper. He is working inside the theory of those machines. That is influence of Turing on Bennett of a kind the citation itself records. It does not make Bennett's three-stage uncomputation a remark about causal loops, and Bennett does not present it as one.

The further step, from Bennett or Turing to process matrices and process functions, is not established by anything read for this draft. Oreshkov, Costa, and Brukner (2012) introduce process matrices for local quantum operations without a predefined causal order. Baumeler and Wolf (2016) study classical process functions and reversible realizations. Those papers have their own citations. This draft has not done the archival or citation-tracing work that would be required to say which lines in them, if any, answer Bennett, and it does not assert that none do. "No influence is claimed" is a limit on this manuscript. It is not a negative historical discovery.

A reconstruction can still be useful if it is marked as one. The useful form is a list of questions one is not entitled to treat as already answered by the earlier text.

From Turing one is entitled to keep the distinction between a number that can be defined and a number whose decimal a machine writes; the picture of computation as successive complete configurations under a finite memory; the technical sense of "circular" as a failure to print infinitely many figures; and the universal machine as a machine that follows a description on its tape. One is not entitled to call a cyclic but uniquely solvable wiring a circular machine, or to call a finite empty search a non-computable number.

From Bennett one is entitled to keep logical irreversibility as the lack of a single-valued inverse of the transition function; the three-stage pattern of history, copy, and uncomputation; the criterion of non-overlapping ranges; and the permission to retain the input when the input is not a function of the output. One is not entitled to treat the canonical lift as that construction, or to treat unitarity of a closed loop as the erasure principle Bennett attributes to Landauer.

From the later mathematics one is entitled to keep the universal quantifier over local replacements; the equivalence with the canonical unitary lift; the existence of the bare permutation for wirings that are not process functions; and the fact that a known finite witness is searchable. One is not entitled to present these statements as what Turing or Bennett proved, or as the next step they left open in those words.

8 Conclusion

The three texts can be placed in one bibliography. They cannot be placed on one line without changing what they say. Turing gives a theory of what a limited memory can write down by successive moves, and separates that notion from definability. Bennett shows how to keep a single-valued inverse at every move of such a machine, by remembering the history and then uncomputing it, and he draws a thermodynamic moral from Landauer that this paper does not re-derive. The process-function theorem says when a wiring remains consistent under every local replacement, and when a particular closed-loop operator is unitary. Consistency of that kind is not circle-freeness, and the unitary lift is not Bennett's reversible emulation.

The historical gain in keeping them apart is negative and definite. It blocks a style of exposition in which a cyclic equation is called circular because Turing used the word, a unitary dilation is called a reversible computer because Bennett used the words, and a finite search is called a universal machine because Turing built one. Each of those sentences sounds continuous and is false to the text it cites. A rational reconstruction that stops at the false sentence is not history. A reconstruction that says which question each text answers, and which later question it does not, is as much history as the documents read here will support. Claims of influence beyond Bennett's explicit use of the Turing machine would need documents this draft does not have.

References

Baumeler, Ä., and S. Wolf. 2016. "The Space of Logically Consistent Classical Processes without Causal Order." *New Journal of Physics* 18:013036. https://doi.org/10.1088/1367-2630/18/1/013036.

Bennett, C. H. 1973. "Logical Reversibility of Computation." *IBM Journal of Research and Development* 17(6):525–532. https://doi.org/10.1147/rd.176.0525. Read from the IBM printing.

Oreshkov, O., F. Costa, and Č. Brukner. 2012. "Quantum Correlations with No Causal Order." *Nature Communications* 3:1092. https://doi.org/10.1038/ncomms2076.

Stinespring, W. F. 1955. "Positive Functions on C^*-Algebras." *Proceedings of the American Mathematical Society* 6:211–216. https://doi.org/10.1090/S0002-9939-1955-0069403-4.

Turing, A. M. 1936. "On Computable Numbers, with an Application to the Entscheidungsproblem." *Proceedings of the London Mathematical Society* s2-42:230–265. https://doi.org/10.1112/plms/s2-42.1.230. Received 28 May 1936, read 12 November 1936. Read from the Society's printing. A later correction by Turing is not used, because it was not re-read for this draft.

Note on assistance. A Grok Bot agent in Cursor generated draft prose beyond spelling checks. The tool version is not recorded. The agent is not an author. Human verification of quotations against the printed pages is pending; the quotations were checked against text extracted from the PDFs named above, which contain ordinary scan errors, and only strings that were legible in that extraction are quoted. The official guide for authors on the Elsevier site returned an access error, so this note's placement is not claimed to satisfy a rule that was not retrieved. No journal submission has been made.[1]

[1]The same disclosure is repeated here because anonymous-review practice and publisher AI rules were not both available. Funding, affiliation, and conflicts are unknown and are not filled in as "none."